



**MODULE 1.3 JAVA, TOMCAT, AND IDE INSTALLATION WITH JSP APPLICATION
VERIFICATION**

**ELVIA GISELLA ADAIR
STUDENT**

**CHRISTINE MAZHATA
TEACHER**

JUNE 14, 2026

OPERATION INVERSION AT LINKEDIN (2011)

Chapter 6 of *The DevOps Handbook* is about slowing things down a bit before speeding things up. The authors focus on the idea that before an organization can improve how it delivers software, it needs to clearly understand how work moves through the system from the moment an idea is requested to the moment it reaches the customer.

The case study in this chapter using organizations like Nordstrom as an example shows that most companies don't have a clear picture of this flow. Work tends to get stuck in different places: waiting for approvals, bouncing between teams, or sitting in queues because priorities keep changing. When teams finally map everything out step by step, they often realize the real problem isn't the developers or operations teams it's the system itself. One of the biggest turning points in the case study is when different groups development, operations, security, and business sit together and draw out the full process. That "visual map" makes the hidden delays obvious. Suddenly, everyone can see where time is being wasted, and it's usually not where people assume it was.

The authors also highlight something simple but powerful: you can't improve what you can't see. Once organizations start measuring things like lead time (how long something takes from start to finish), they begin to understand where the real bottlenecks are. And instead of trying to fix everything at once, they can focus on the parts that are actually slowing things down the most.

Another important idea is that improvement work needs time and focus. If people are expected to fix processes on top of their normal workload, nothing really changes. So the book emphasizes giving teams space and ownership to improve the system, not just maintain it.

The main lesson from this chapter is pretty straightforward: most performance problems aren't individual problems they're system problems. And the only way to solve them is by making the work visible, understanding the flow, and continuously improving it together as one team.